

Poznan University of Technology
Faculty of Computing
Institute of Computing Science

Master's thesis

**INTERPRETABLE METHODS FOR DATA-TO-TEXT
GENERATION**

Dominik Maćkowiak, 151915

Supervisor
Mateusz Lango, PhD

Poznań, 2026

Abstract

Contents

1	Introduction	1
1.1	Motivation	1
1.2	Thesis structure	1
2	Data-to-Text Generation	2
2.1	Problem definition	2
2.2	Methods	2
2.3	Interpretable D2T Approaches	2
2.4	Performance Evaluation	2
3	Building interpretable meaning representations from complex data	3
3.1	Method Overview	3
3.2	Data conversion with LLMs	3
3.2.1	Text-First Extraction with Normalization	3
3.2.2	Blueprint-Iterative Refinement	4
3.2.3	Evolving Catalog	6
3.3	Fine-tuned model for data conversion	7
4	Surface realization — evolving programs using AlphaEvolve	8
4.1	Modern approaches for writing complex programs	8
4.2	Methods overview	8
4.3	OpenEvolve	8
4.4	Program database	8
4.5	Prompt and selection	8
4.6	Evaluation	8
5	Experiments	9
5.1	Meaning representation	9
5.2	Surface realization	9
5.3	Whole system evaluation	9
6	Discussion	10
7	Summary	11
	Bibliography	12
A	Prompts	13
A.1	Text-First Extraction with Normalization	13

A.2	Blueprint-Iterative Refinement	13
A.3	Evolving Catalog	14

Chapter 1

Introduction

1.1 Motivation

1.2 Thesis structure

The structure of the thesis is as follows. Chapter 2 provides an overview of data-to-text generation. Chapter 3 describes the methods for building interpretable meaning representations from complex data. Chapter 4 presents the surface realization approach using OpenEvolve. Chapter 5 reports the experimental results. Chapter 6 discusses the findings, and Chapter 7 concludes the thesis.

Chapter 2

Data-to-Text Generation

2.1 Problem definition

2.2 Methods

2.3 Interpretable D2T Approaches

2.4 Performance Evaluation

Chapter 3

Building interpretable meaning representations from complex data

3.1 Method Overview

3.2 Data conversion with LLMs

Input data format. All three methods operate on a collection of *raw structured instances*. A raw structured instance is a single JSON object representing one self-contained data record, such as a weather forecast for a specific city, a product description, or a sports event summary. Each instance contains nested fields with structured data (e.g., time series, categorical values, numerical measurements) that must be converted into semantic triples. For example, a weather forecast instance might contain fields for city name, date, temperature values, wind speed, and weather conditions. Let $D = \{d_1, \dots, d_N\}$ denote the set of N such instances extracted from the input JSON.

3.2.1 Text-First Extraction with Normalization

Text-First Extraction with Normalization decomposes the data-to-triples problem into two fully batched LLM stages and a normalization stage. The method deliberately avoids any inter-instance coupling during extraction: each reference and each triple set is produced independently and in parallel, which makes the method stateless, at the cost of not enforcing predicate consistency during extraction.

Stage 1 — Reference text generation. In the first stage the model is invoked once per instance under a **text-generation system prompt**, P_{txt} (see Appendix A.1), whose objective is to convert one structured data instance into concise natural language and to return the result as a JSON object of schema `{"text": "..."}1. The prompt further instructs the model to capture the most important information, trends, extremes, and notable conditions, and to summarize time series (e.g., weather forecasts) at a high level rather than listing every data point.`

All N requests are packaged as a single OpenAI Batch API job and a single request outputs t_i , where t_i is the generated reference text for instance d_i . This batched stage yields, for every instance, a natural-language reference t_i that constitutes the intermediate representation from which triples are subsequently derived.

¹Throughout this work, JSON schemas are presented in simplified notation with placeholder values (e.g., `"text": "..."`) for readability, rather than as formal JSON Schema definitions.

Stage 2 — Triples-from-text extraction. A second OpenAI Batch is then assembled in which every reference t_i is paired with a **triples-from-text system prompt**, P_{tr} (see Appendix A.1), whose objective is to extract semantic triples from natural language text, returning JSON of schema `{"triples":["subject":"...", "predicate":"...", "object":"..."]}`. The prompt imposes two constraints: it instructs the model to preserve the full meaning of the input text and to keep predicates in `lower_snake_case` when possible, and to avoid duplicate triples. Each instance is therefore an independent text-to-triples problem, and the batch can be fully parallelized. The output of a single request is T_i , where T_i is a set of triples that describe the reference text t_i .

Stage 3 — Post-hoc predicate normalization. Because Stage 2 imposes no inter-instance vocabulary control, this stage invokes a predicate-normalization routine. The routine operates on the flattened list of triples and iteratively merges semantically equivalent predicates:

Let Π be the set of distinct predicates present in the extracted triples. A Union-Find structure over Π is initialized with every predicate in its own singleton class. A *Union-Find* (disjoint-set) is a data structure that maintains a collection of disjoint sets and supports two operations: **find**(x) returning the representative element of the set containing x , and **union**(a , b) merging the sets containing a and b . A *singleton class* is an equivalence class containing only a single element.

For each round $r = 1, 2, \dots$ (bounded by $2|\Pi|$):

- The model is prompted with the current set of canonical predicates (class representatives) and asked to propose candidate equivalent pairs under a **candidate-pairs system prompt** (see Appendix A.1).
- If no candidate pairs are returned, or if the candidate set is identical to one proposed in an earlier round (indicating the model has converged or is cycling without progress), the loop terminates.
- Otherwise, every candidate pair is sent to the model under a **strict-equivalence system prompt** (see Appendix A.1) that returns a JSON object with three fields: **equivalent** (boolean), **confidence** (a score between 0 and 1), and **reason** (a textual explanation). The model is instructed to judge equivalence strictly (mergeable in a knowledge graph), not mere relatedness.
- Pairs judged equivalent are merged in the Union-Find structure. If a round produces no merges, the loop terminates.

After termination, each predicate is mapped to the canonical representative of its class, defined as the first member of the class (deterministic, order-based choice). The triples are then rewritten with these canonical predicates.

In the normalization step, the final output combines: (i) generated references, (ii) extracted triples, and (iii) normalized triples alongside the canonical mapping and the comparison audit trail.

3.2.2 Blueprint-Iterative Refinement

Blueprint-Iterative Refinement introduces an explicit notion of extraction blueprints that act as an intermediate, human-inspectable artifact between the domain declaration and the final triple extraction. An *extraction blueprint* is a structured specification containing predicate labels, example triples, example text, and transformation guidelines. The blueprints are generated top-down from the domain name, then iteratively validated and revised against the generated references

until it is confirmed that they can extract triples for every instance. Only then is a single batched triple-extraction run executed under the finalized blueprint.

Stage 1 — Initial blueprint generation. Given the user-supplied problem domain string δ (e.g., weather forecast), the model is invoked once under a **blueprint-creation system prompt**, $P_{\text{blueprint}}$ (see Appendix A.2), and asked to design an extraction blueprint for a domain. Its response is constrained by a strict JSON schema to contain exactly four fields:

$$R = (\Pi_R, E, T_{\text{ex}}, G),$$

where Π_R is a list of predicate labels, E is a list of example triples $\{(\sigma_k, \pi_k, \omega_k)\}$ using those predicates, T_{ex} is a short natural-language passage consistent with the example triples, and G is a list of free-text transformation guidelines. The response is sanitized, which deduplicates predicates and guidelines, discards malformed example triples, and falls back to the corresponding field values from the previous blueprint R_{r-1} for any field that is empty or malformed (for the initial blueprint R_0 , empty fields default to empty lists or strings). The resulting R_0 constitutes the initial blueprint. Conceptually, R_0 is a self-contained specification of how the model believes instances of the domain should be verbalized and triplelized, complete with concrete formatting conventions embedded in the example triples.

Stage 2 — Reference text generation. The reference-generation batch is identical to that of the previous method: every instance d_i is sent to the **text-generation system prompt** P_{txt} within a single OpenAI Batch, producing references t_i .

Stage 3 — Blueprint validation and iterative revision. The core of the method is an iterative loop, bounded by R_{max} . The loop maintains a current blueprint R and operates as follows for each round $r = 1, \dots, R_{\text{max}}$:

1. **Feasibility scan.** A random reference text with replacement t_i is selected and the model is invoked under a **blueprint-feasibility system prompt**, P_{feas} (see Appendix A.2), which receives the current blueprint R and the text t_i , and returns a JSON object with three fields: **possible** (boolean indicating whether the blueprint is sufficient), **reason** (textual explanation), and **blueprint_gaps** (a list of descriptions of what is missing from the blueprint in order to extract triples from t_i).
2. **Pass condition.** If the scan reaches the R_{max} without encountering a **possible = false** judgment, the blueprints are declared validated against all texts and the loop terminates.
3. **Failure and revision.** As soon as a reference t_{i^*} is judged infeasible under R , the scan halts. The **blueprint-revision system prompt**, P_{rev} (see Appendix A.2), is then invoked with the domain δ , the current blueprint R , the failing text t_{i^*} , the failure reason, and the enumerated **blueprint_gaps**. Its response is again constrained to the same four-field schema structure $R = (\Pi_R, E, T_{\text{ex}}, G)$ described in Stage 1. The model is instructed to keep existing high-quality blueprint components, but add or adjust what is necessary to cover the failing case. The sanitized result replaces R as the new current blueprint.
4. **Restart.** Because revising the blueprint may break previously validated instances or expose new gaps, the scan is restarted and the round number r set to 1. The loop thus monotonically accumulates coverage, but backtracks the scan position whenever the blueprint is modified.

If the budget R_{max} is exhausted before all texts are validated, the method emits a warning and proceeds to final extraction with the best-effort blueprint; the output records that validation was incomplete.

This stage implements a generate-test-revise loop in which the blueprint is the artifact being refined, the reference texts act as test cases, and the feasibility prompt acts as a model-internal test oracle.

Stage 4 — Final batched extraction under finalized blueprint. Once the validation loop has terminated, a single OpenAI Batch is submitted in which every reference t_i is paired with a `triples-from-text-with-blueprint` system prompt, $P_{\text{tr}\&R}$ (see Appendix A.2). This prompt instructs the model to extract semantic triples from the natural language text using the provided blueprint, to follow the blueprint guidelines, to mimic the formatting style of the example triples, and to prefer predicates from the blueprint predicate list whenever possible. The blueprint R is serialized and injected in each request. The batch output is parsed into T_i .

3.2.3 Evolving Catalog

Evolving Catalog is a hybrid text-first data-to-triples pipeline that combines an evolving predicate catalog with a stability-triggered transition from sequential to batch processing. It is designed to keep the predicate vocabulary consistent across instances while reducing the latency overhead of per-instance, order-dependent LLM calls.

Stage 1 — Reference text generation. An OpenAI Batch is assembled in which every instance d_i is paired with the `text-generation` system prompt P_{txt} (see Appendix A.3), whose purpose is to produce a concise natural-language summary that captures the most important information, trends, extremes, and notable conditions, and summarizes time series at a high level. The batch output is parsed into text reference t_i .

Stage 2 — Sequential catalog-driven triple extraction. The second stage maintains a mutable predicate catalog C , initialized as empty. The catalog is a set of entries $\{(\pi, e_\pi)\}$, where each entry associates a predicate label π with an example triple $e_\pi = (s, \pi, o)$ in which that predicate was first observed. The catalog thus carries both a normalized vocabulary and a small set of usage examples that serve as in-context guidance for the extraction model.

The instances are processed strictly in order:

1. **Bootstrap instance** ($i = 0$). Because C is empty, the reference t_0 is sent to the model under a `first-instance` system prompt, P_{first} (see Appendix A.3), that simply asks for semantic triples from the text with concise `lower_snake_case` predicates and no duplicates. This yields a set of triples T_0 .
2. **Catalog-guided instances** ($i \geq 1$). For each subsequent instance d_i , the reference t_i is sent under a `catalog-guided` system prompt, P_{cat} (see Appendix A.3), in which the current catalog C is serialized and injected into the user prompt. The model is instructed to use only predicates from the provided catalog when possible and to introduce a new predicate only when none of the catalog predicates can describe the fact. This yields triples T_i .

After each instance, every triple in T_i is inspected. If a triple contains a predicate $\pi \notin C$, a new entry (π, e_π) is added to C , where e_π is set to that triple (the first occurrence).

Stage 3 — Stable-window heuristic and batched tail processing. A counter w tracks the number of consecutive instances that did not introduce any new predicate; it is reset to zero whenever a new predicate is added. Let W denote the stable window parameter. The sequential phase terminates as soon as $w \geq W$ and at least one instance remains unprocessed. Let i^* be the index at which this condition is first met; the catalog C at that point is declared frozen.

All instances $d_{i^*+1}, \dots, d_N$ (the tail) are then processed in a single second OpenAI Batch, again under P_{cat} and with the frozen catalog C injected into every request. Two design choices are important here:

- The tail is parallelized, hence the cost of the remaining $N - i^* - 1$ LLM calls is amortized into one batch.
- Predicates that first appear within the tail are not added back to C . The catalog is finalized at the moment of stability, which guarantees that the predicate vocabulary presented to a human evaluator is fully determined by the sequential phase and does not silently grow during batch processing.

3.3 Fine-tuned model for data conversion

Chapter 4

Surface realization — evolving programs using AlphaEvolve

4.1 Modern approaches for writing complex programs

4.2 Methods overview

4.3 OpenEvolve

4.4 Program database

4.5 Prompt and selection

4.6 Evaluation

Chapter 5

Experiments

5.1 Meaning representation

5.2 Surface realization

5.3 Whole system evaluation

Chapter 6

Discussion

Chapter 7

Summary

Bibliography

Appendix A

Prompts

A.1 Text-First Extraction with Normalization

This section contains the complete text of all system prompts used in the Text-First Extraction with Normalization method (Section 3.2).

Text-Generation System Prompt (P_{txt})

You convert one structured data instance into concise natural language. Return ONLY JSON with schema: `{"text":"..."}`. Capture the most important information, trends, extremes, and notable conditions. If the instance contains a time series (for example a weather forecast), summarize it at a high level instead of listing every point.

Triples-From-Text System Prompt (P_{tr})

You extract semantic triples from natural language text. Return ONLY JSON with this exact schema: `{"triples":[{"subject":"...", "predicate":"...", "object":"..."}]}`. Rules: preserve the full meaning of the input text, avoid duplicate triples, and keep predicates in lower_snake_case when possible.

Candidate-Pairs System Prompt

You find candidate predicate pairs that may be semantically equivalent. Return ONLY JSON with schema: `{"pairs":[{"predicate_a":"...", "predicate_b":"..."}]}`. Rules: include only pairs with high likelihood of strict semantic equivalence; do not include pairs that are merely related; do not include duplicates or self-pairs.

Strict-Equivalence System Prompt

You decide if two predicates are semantically equivalent in meaning. Return ONLY JSON with schema: `{"equivalent": true|false, "confidence": 0.0-1.0, "reason": "..."}`. Be strict: equivalent only if they can be safely merged in a knowledge graph.

A.2 Blueprint-Iterative Refinement

This section contains the complete text of all system prompts used in the Blueprint-Iterative Refinement method (Section 3.2).

Text-Generation System Prompt (P_{txt})

You convert one structured data instance into concise natural language. Return ONLY JSON with schema: {"text":"..."} . Capture the most important information, trends, extremes, and notable conditions. If the instance contains a time series (for example a weather forecast), summarize it at a high level instead of listing every point.

Blueprint-Creation System Prompt ($P_{\text{blueprint}}$)

You design extraction rules for a domain. Return ONLY JSON with schema: {"predicates":["..."], "example_triples":[{"subject":"...", "predicate":"...", "object":"..."}], "example_text":"...", "guidelines":["..."]}. Rules: include precise and reusable predicates, provide concrete example triples with strict formatting, provide an example text matching those triples, and give practical transformation guidelines.

Blueprint-Feasibility System Prompt (P_{feas})

You evaluate whether given extraction rules are sufficient to extract triples from a text. Return ONLY JSON with schema: {"possible":true|false, "reason":"...", "rule_gaps":["..."]}.

Blueprint-Revision System Prompt (P_{rev})

You revise extraction rules so triples can be produced from a text. Return ONLY JSON with schema: {"predicates":["..."], "example_triples":[{"subject":"...", "predicate":"...", "object":"..."}], "example_text":"...", "guidelines":["..."]}. Keep existing high-quality rules, but add or adjust what is necessary to cover the failing case.

Triples-From-Text-With-Blueprint System Prompt ($P_{\text{tr\&R}}$)

You extract semantic triples from natural language text using provided rules. Return ONLY JSON with this exact schema: {"triples":[{"subject":"...", "predicate":"...", "object":"..."}]}. Follow rule guidelines and mimic the formatting style from example triples. Prefer predicates from the rule predicate list whenever possible.

A.3 Evolving Catalog

This section contains the complete text of all system prompts used in the Evolving Catalog method (Section 3.2).

Text-Generation System Prompt (P_{txt})

You convert one structured data instance into concise natural language. Return ONLY JSON with schema: {"text":"..."} . Capture the most important information, trends, extremes, and notable conditions. If the instance contains a time series (for example a weather forecast), summarize it at a high level instead of listing every point.

First-Instance System Prompt (P_{first})

You extract semantic triples from text. Return ONLY JSON with this exact schema: {"triples":[{"subject":"...", "predicate":"...", "object":"..."}]}. Use concise predicate labels in lower_snake_case when possible and avoid duplicates.

Catalog-Guided System Prompt (P_{cat})

You extract semantic triples from text using a predicate catalog with examples. Return ONLY JSON with this exact schema: {"triples":[{"subject":"...", "predicate":"...", "object":"..."}]}. Use only predicates from the provided catalog when possible. Introduce a new predicate only when none of the catalog predicates can describe the fact.



© 2026 Dominik Maćkowiak

Poznan University of Technology
Faculty of Computing
Institute of Computing Science

Typeset using L^AT_EX in Computer Modern.

BibT_EX:

```
@mastersthesis{ key,  
  author = "Dominik Maćkowiak",  
  title = "{Interpretable methods for data-to-text generation}",  
  school = "Poznan University of Technology",  
  address = "Pozna{\'}n, Poland",  
  year = "2026",  
}
```